A3-PS013-TC015

Incident Timeline & Evidence Collection for Small Teams

A structured, tamper-evident workflow for collecting, correlating and reviewing digital evidence

Barath C G, Sri Ramanarayana S N

Faculty mentor: Dr. Rakoth Kandan Sambandam

barath.cg@btech.christuniversity.in

sri.ramanarayana@btech.christuniversity.in

rakoth.kandan@christuniversity.in

AIORI Hackathon 3 · Problem Statement 013

Contents

What this presentation covers

3 Problem Description

4 Background

5 Literature & RFC Review

6 Proposed Solution

7 PS-013 Requirement Coverage

8 Optimization Proposed

9 Solution Architecture & Design

10 Data Flow Diagram

11 Synthetic Incident Scenarios

12 Pseudocode & Demo Plan

13 Deployment Timeline

14 Bibliography

15 Appendix A1 · Worked Example

16 Appendix A2 · Validation Plan

17 Appendix A3 · Evaluation & Triage Baseline

18 Appendix A4 · Limitations & Security

19 Thank You

Problem Description

Small teams rebuild incident timelines by hand from scattered logs

- **Evidence is scattered.** Authentication, application, endpoint and network logs sit in different files and systems.
- **Time is unreliable.** Mixed time zones, formats and clock offsets make the true order of events hard to trust.
- **Integrity is uncertain.** Copied files and manual notes lose provenance, so later verification is difficult.
- **Reports are hard to reproduce.** Manual summaries cannot easily be re-created or audited by a second reviewer.

Who is affected

- Small IT and security teams without an enterprise SIEM or forensic suite
- Incident responders who need one chronological view
- Auditors and reviewers who need traceable sources

Why it matters

Slower triage, missed links between events, and reports that cannot be audited

Typical case: a team has an auth log, an application log and a proxy log in three formats and two time zones, and must decide what happened first.

Goal: make incident evidence easier to collect, preserve, correlate and explain, without replacing human investigation.

Background

Evidence must be preserved first, and timestamps must be made comparable

- **Evidence handling.** Collect from most to least volatile; record who collected what and when, and restrict access [2].
- **Log management.** Consistent, protected logs are the raw material of an investigation [3]; synchronize clocks and detect tampering [9].
- **Forensics in incident response.** IT-focused forensic techniques fit inside the incident-response lifecycle [1][6].
- **Time is the hard part.** RFC 3339 requires an explicit UTC offset [4]; RFC 3227 says to record clock drift and whether a time is UTC or local [2]; syslog marks whether the zone is known [5].

One instant, four representations

Synthetic example

Representation	Value
syslog-style: no year, no zone	Mar 14 19:42:10
Local time with offset	2026-03-14 19:42:10 +05:30
Unix epoch seconds	1773497530
RFC 3339, UTC	2026-03-14T14:12:10Z

All four are **14:12:10 UTC**. We keep the original value and store one normalized UTC value; if the zone is unknown, the event is flagged as an assumption.

Literature & RFC Review

Standards define the practice; existing tools solve adjacent problems

Standards we build on

- **RFC 3227 [2].** Order of volatility, chain of custody, record clock drift
- **NIST SP 800-86 [1].** Forensic techniques inside incident response
- **NIST SP 800-92 [3].** Practical log-management processes
- **RFC 3339 / 5424 [4][5].** Explicit UTC offsets; syslog timestamp and zone-known flag
- **OWASP Logging [9].** Synchronize time; build in tamper detection

Existing approaches and the gap

Approach	Focus (per project documentation)	Gap this project targets
Plaso / log2timeline [7]	Python engine that builds super timelines from many sources	We do not match its parser coverage; we add intake manifest and review for small teams
Timesketch [8]	Collaborative timeline analysis: sketches, tags, annotations	Focus on evidence intake, integrity hashing and a reviewable report
Enterprise SIEM	Continuous monitoring and alerting at scale	Typically more setup than one small investigation needs
Spreadsheets and notes	Flexible, no setup	No provenance or hashes; hard to reproduce

Positioning: an integrity-first path from raw logs to a reviewable timeline. The difference is scope and workflow, not a new algorithm. *Qualitative comparison.*

Proposed Solution

A lightweight workbench: preserve first, then normalize, correlate and review

1 Ingest	2 Preserve	3 SHA-256	4 Parse	5 Normalize	6 Correlate	7 Timeline	8 Export
Import log files; record source, collector and time	Copy each original to a read-only store; never edited	Hash every original; write the evidence manifest	One parser per format; bad lines logged as gaps	Convert to UTC; keep original; flag assumptions	Order, group and de-duplicate events	Search by time, source, type and keyword	Re-verify hashes; export CSV / JSON report

Key capabilities

- Originals stay read-only; normalized copies are stored separately
- Every event keeps its source file, line, parser and original value
- SHA-256 at intake; re-verified before export, mismatches flagged
- Parsing gaps and unknown time zones are recorded, never hidden
- Search and filter by time, source, event type and keyword
- CSV and JSON incident report with manifest and gap list

Optional AI assistance

planned, not implemented

- Draft summary of an analyst-selected time window
- Suggest candidate event relationships, with the reason
- Always labeled as a draft, never a forensic conclusion
- The core pipeline works without it

Status: design stage. Nothing described here is implemented yet; no measured results are claimed in this deck.

PS-013 Requirement Coverage

Every PS-013 requirement is mapped to a planned feature; none is implemented yet

#	PS-013 requirement	Planned approach	Slide	Status
1	At least two log formats	auth.log (syslog) and app.log (JSON lines); CSV and proxy logs targeted	9, 15	Planned
2	Three synthetic incident scenarios	S1 failed logins then success; S2 role change; S3 unusual outbound transfer	11	Planned
3	Originals preserved, normalized copies	Read-only store D1; normalized events kept separately in SQLite D3	6, 10	Planned
4	Evidence hashes	SHA-256 with hashlib at intake; manifest D2	6, 10	Planned
5	Time zones and clock skew	UTC conversion, declared offsets, per-source skew entry, assumption flags	4, 15	Planned
6	Parsing gaps documented	Gap log with file, line number and reason; shown in report	10, 16	Planned
7	Manual triage baseline	Same scripted questions answered manually and with the tool, timed	17	Planned
8	Log ingestion and parsing tool	Python parser modules, one per format	9, 12	Planned
9	Timestamp normalization	Parse to UTC (RFC 3339); keep original value	4, 12	Planned
10	Evidence preservation and hashing	Read-only copy, manifest, re-verification before export	10, 12	Planned
11	Searchable incident timeline viewer	Streamlit: filter by time, source, type and keyword	9, 10	Planned
12	Incident report export	CSV and JSON with manifest, gaps and integrity status	10, 12	Planned
13	Prototype demo and pseudocode	Live demo on synthetic logs; pseudocode on slide 12	12	Planned
14	GitHub collaboration and submission	Shared repository, issues and pull requests, README, deck PDF, video	13	Planned

Status reflects the design stage: no feature above has been built or tested.

Optimization Proposed

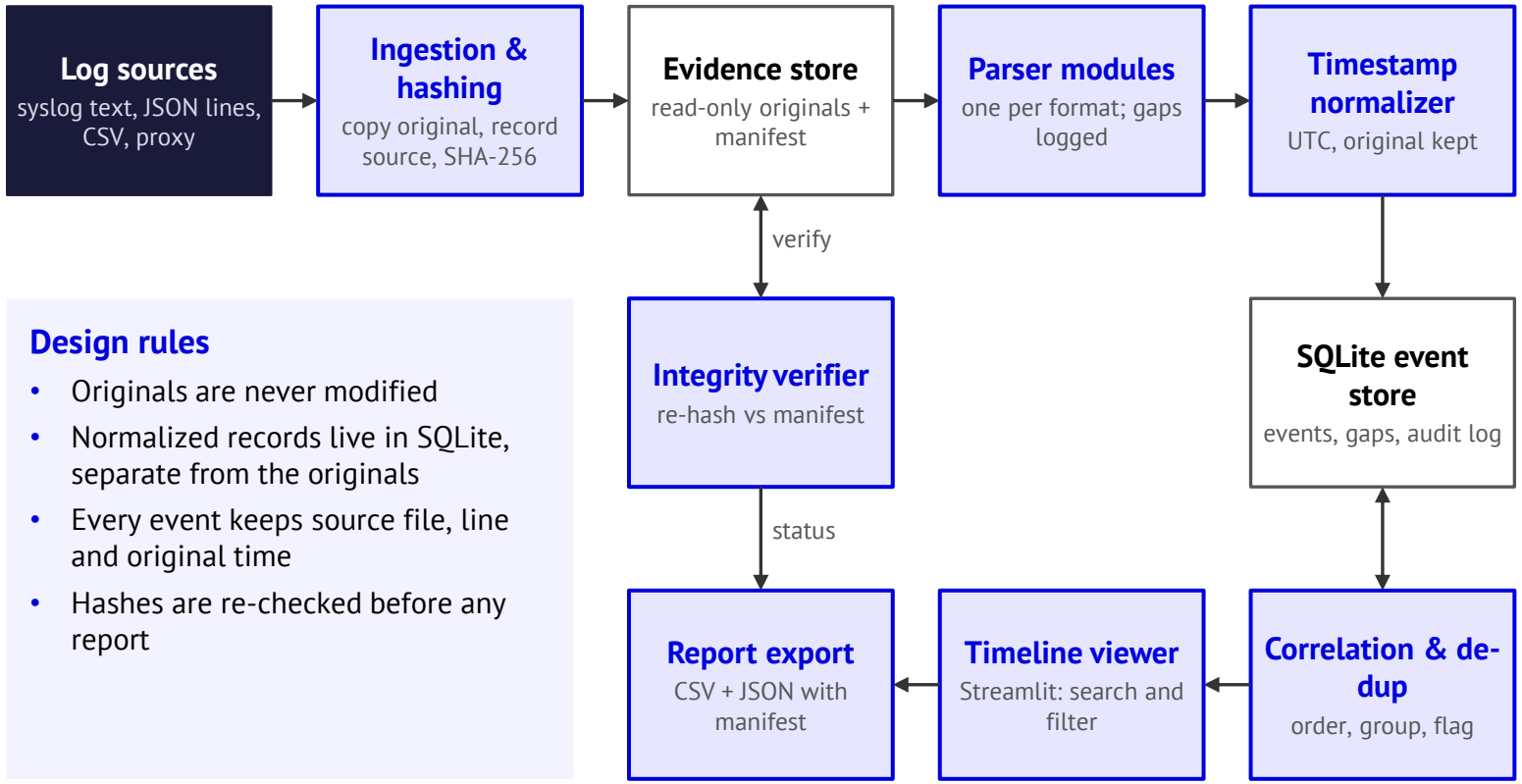
Each optimization targets a specific piece of manual work

Area	Before: typical manual workflow	After: proposed workflow	How it will be measured
Evidence intake	Logs copied ad hoc; source and collection time often unrecorded	Import records source, collector, time and metadata	Share of events with complete provenance
Timestamps	Formats and zones reconciled by hand	Parsed to UTC; original and zone assumption kept	Correct UTC order on mixed-format, mixed-zone test set
Integrity	Changes to copied files may go unnoticed	SHA-256 at intake, re-verified before reporting	Modified test file is detected (SHA-256 mismatch)
Event noise	Repeated records clutter the review	Duplicates detected and grouped, not deleted	Duplicate reduction on synthetic logs
Triage	Searching across separate files	Indexed, searchable timeline: time, source, type, keyword	Manual vs tool-assisted triage time on scripted questions
Reporting	Manual notes; hard to reproduce	Manifest and timeline exported from the same data	Same inputs give the same report
AI summary (optional)	Not applicable	Draft summary, labeled, analyst-validated	Unsupported statements per summary

No performance gains are claimed. Measurements will be taken on synthetic incidents during testing.

Solution Architecture & Design

A read-only evidence store feeds parsing, verification and reporting



Design rules

- Originals are never modified
- Normalized records live in SQLite, separate from the originals
- Every event keeps source file, line and original time
- Hashes are re-checked before any report

Figure: Proposed system architecture, component view (design stage, not yet implemented)

Technology stack

proposed, to be confirmed

- **Python 3 + hashlib.** SHA-256 is built in [10]
- **SQLite.** Single-file store; tables for events, manifest, parsing gaps
- **Parser modules.** One per format; two required, four targeted
- **UTC, RFC 3339.** Common time reference [4]
- **Streamlit.** Local, searchable timeline dashboard
- **CSV and JSON.** Incident report and manifest export
- **AI layer (optional).** Not selected; no real logs sent to external services

Change from earlier draft: the interface is now Streamlit, and PDF export is out of core scope (CSV and JSON only).

Data Flow Diagram

Evidence is hashed and stored before it is parsed, and every step is logged

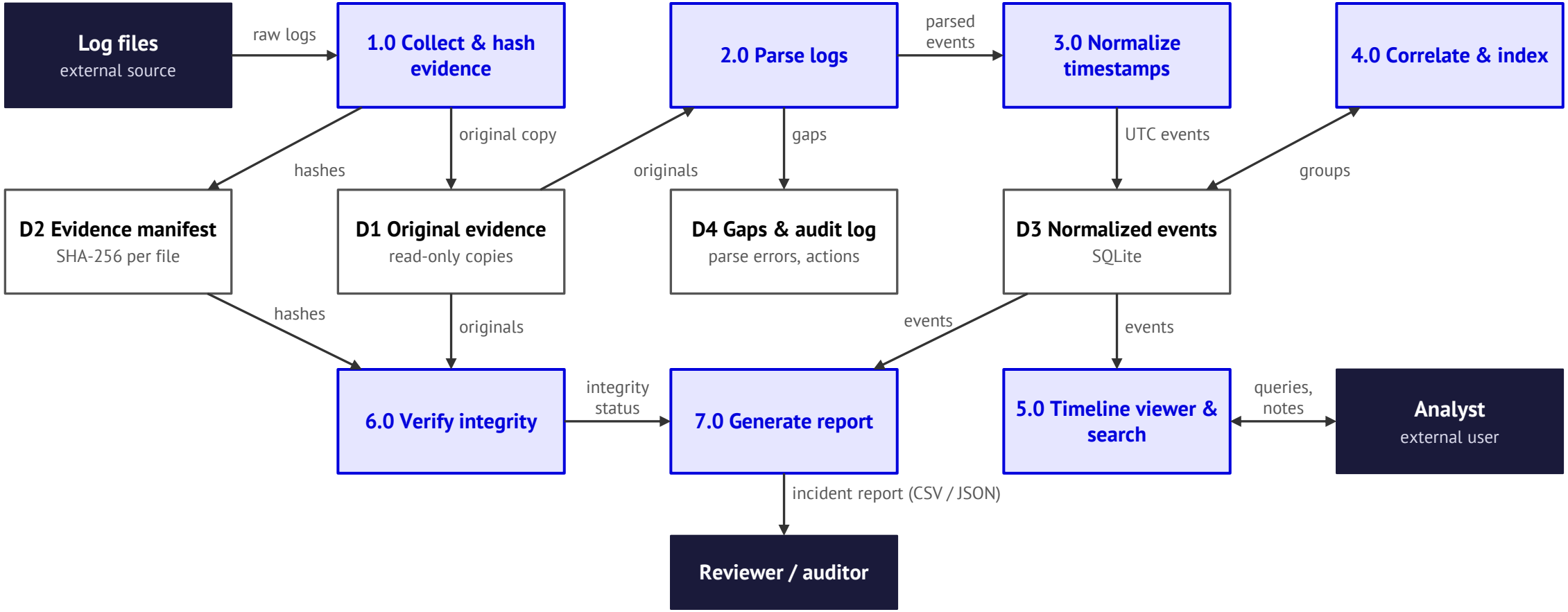


Figure: Level-1 Data Flow Diagram (proposed). Dark = external entity · blue = process · open box = data store

Every process writes to D4 (links omitted for clarity). Normalized events (D3) never overwrite originals (D1). The optional AI assistant, if built, reads only analyst-selected events.

Synthetic Incident Scenarios

Three planned test cases, each built from generated logs

S1 · Failed logins, then success

Sources: auth.log (syslog text), app.log (JSON lines)

Planned events: repeated failed logins for one account from a documentation IP, then a successful login

Stresses: year-less syslog time, declared zone, repeated lines

Expected timeline: failures grouped, then the success, in UTC order

S2 · Privilege escalation

Sources: app.log (JSON audit event), endpoint.csv

Planned events: role change from user to admin shortly after the S1 login; a tool launch

Stresses: CSV timestamp with no zone, mixed date formats

Expected timeline: role change after the login; zone assumption flagged

S3 · Unusual outbound transfer

Sources: proxy.log (epoch seconds), plus a file with malformed lines

Planned events: large upload to a documentation IP after the role change

Stresses: epoch time, malformed and duplicate lines

Expected timeline: upload placed last; bad lines listed as parsing gaps

All data is generated by the team using documentation IP ranges (RFC 5737). These are planned test cases, not observed incidents. Observed results are pending (see Appendix A2).

Pseudocode & Demo Plan

Core pipeline in pseudocode, and the demonstration we plan to give

Pseudocode (proposed design, not implemented)

```
for file in input_files:
    original = copy_read_only(file)          # D1
    manifest[file] = sha256(original)        # D2
    fmt = detect_format(original)
    for line_no, line in lines(original):
        try:  ev = parsers[fmt](line)
        except: log_gap(file, line_no)       # D4
        ev.utc = to_utc(ev.raw_time, tz, skew)
        if tz unknown: ev.flag = 'assumed'
        if seen(ev.key): group_duplicate(ev)
        else: store(ev)                      # D3
timeline = query(time, source, type, keyword)
for file in manifest:                        # before export
    if sha256(file) != manifest[file]:
        status[file] = 'MISMATCH'
export(timeline, manifest, gaps, status)     # CSV, JSON
```

Demo plan (31 Oct to 4 Nov)

1. Import the mixed-format synthetic logs (S1 to S3)
2. Show the manifest and SHA-256 hashes
3. Show the UTC timeline with assumption flags
4. Search and filter by time, source and keyword
5. Change a copied file; verification reports a mismatch
6. Show the parsing-gap list from malformed lines
7. Export the CSV and JSON incident report

Status: planned. The demo will run live on the synthetic data in the repository; no demo has been recorded yet.

Deployment Timeline

Four phases, aligned to the AIORI-3 schedule

Level 1 package

Now to 14 Oct

- Presentation PDF in repository
- Team introduction video link
- Repository transferred to AIORI account

Core pipeline

16 to 22 Oct

- Event schema; at least 2 log formats (target 4)
- Ingestion, read-only store, SHA-256 manifest
- UTC normalization; parsing-gap log

Timeline and report

23 to 30 Oct

- Correlation and de-duplication
- Streamlit searchable timeline
- Integrity re-check; CSV / JSON report export

Test and demo

31 Oct to 4 Nov

- 3 synthetic scenarios; manual vs tool triage timing
- Defect fixes; README and setup steps
- Live demo and pseudocode walkthrough

Method: short iterations; GitHub issues, branches and pull requests from both members; mentor review in the scheduled sessions (16 to 31 Oct); a synthetic test set in the repository. Submission: GitHub repository with README, presentation PDF, introduction video and demo, per the AIORI-3 guidelines [11].

Status: completed so far: problem analysis, literature review, proposed architecture and data flow (this deck). Planned: every implementation, test and demo item above. No test has been run.

Team responsibilities

Barath C G
Ingestion, parsing, timestamp normalization

Sri Ramanarayana S N
Evidence integrity, timeline view, reporting

Dr. Rakoth Kandan
Faculty mentor: guidance, review, validation

Bibliography

[1] K. Kent, S. Chevalier, T. Grance, H. Dang. *Guide to Integrating Forensic Techniques into Incident Response*, NIST SP 800-86, 2006. csrc.nist.gov/pubs/sp/800/86/final

[2] D. Brezinski, T. Killalea. *Guidelines for Evidence Collection and Archiving*, RFC 3227 (BCP 55), 2002. rfc-editor.org/rfc/rfc3227

[3] K. Kent, M. Souppaya. *Guide to Computer Security Log Management*, NIST SP 800-92, 2006. csrc.nist.gov/pubs/sp/800/92/final

[4] G. Klyne, C. Newman. *Date and Time on the Internet: Timestamps*, RFC 3339, 2002. rfc-editor.org/rfc/rfc3339

[5] R. Gerhards. *The Syslog Protocol*, RFC 5424, 2009. rfc-editor.org/rfc/rfc5424

[6] A. Nelson, S. Rekhi, M. Souppaya, K. Scarfone. *Incident Response Recommendations and Considerations for Cybersecurity Risk Management: A CSF 2.0 Community Profile*, NIST SP 800-61 Rev. 3, 2025. csrc.nist.gov/pubs/sp/800/61/r3/final

[7] Plaso (log2timeline) project. github.com/log2timeline/plaso

[8] Timesketch: collaborative forensic timeline analysis. github.com/google/timesketch

[9] OWASP Logging Cheat Sheet. cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

[10] Python documentation: hashlib, secure hashes and message digests. docs.python.org/3/library/hashlib.html

[11] AIORI-3 Hackathon Guidelines (milestones and submission steps). portal.aiori.in/aiori-3/hackathon-guideline/

Appendix A1 · Worked Example

Synthetic example: four log files become one ordered, traceable timeline

Illustration with synthetic data (documentation IP addresses). Not output from an implemented system. Scenarios S1 to S3 are defined on slide 11.

Source	Original timestamp	Normalized (UTC)	Event	Flag
auth.log	Mar 14 19:42:10	2026-03-14T14:12:10Z	5 failed logins: svc-backup from 203.0.113.45	Year taken from collection metadata; zone +05:30 declared
auth.log	Mar 14 19:42:41	2026-03-14T14:12:41Z	Successful login: svc-backup	Same as above
app.log	2026-03-14T14:13:02Z	2026-03-14T14:13:02Z	Role change: svc-backup to admin	None
endpoint.csv	03/14/2026 02:13:40 PM	2026-03-14T14:13:40Z	Archive tool launched	Zone unknown: UTC assumed, analyst to confirm
proxy.log	1773497645	2026-03-14T14:14:05Z	Large upload to 198.51.100.7	None

At import

Each file is hashed (SHA-256) and listed in the manifest with source and collection time. Every row above keeps its source file and original value.

Illustrative draft summary *unverified until analyst review*

Failed logins on svc-backup were followed by a successful login, an admin role change and a large upload within about two minutes.

Appendix A2 · Validation Plan

Planned tests: expected outcomes defined, observed results still to be recorded

ID	Test (synthetic data)	Expected outcome	Observed
T1	Mixed log formats and time zones	Events in correct UTC order; originals retained	Pending
T2	Event with no zone; known clock skew	Flagged as assumption; skew correction recorded	Pending
T3	Original-file preservation	Stored original's SHA-256 equals the source hash; originals never written	Pending
T4	Evidence file modified after collection	SHA-256 mismatch reported before export	Pending
T5	Malformed and unsupported lines	Logged as gaps with file and line; valid lines still processed	Pending
T6	Repeated log lines	Duplicates grouped and counted; originals kept	Pending
T7	Scenario S1: failed logins, then success	Failures and success appear in correct order	Pending
T8	Scenario S2: role change	Role change placed after login; zone assumption flagged	Pending
T9	Scenario S3: outbound transfer	Upload placed last; bad lines listed as gaps	Pending
T10	Manual vs tool-assisted triage	Times recorded per scenario (see A3)	Pending
T11	Optional AI draft vs ground truth	Unsupported statements counted; labeled draft	Pending

Observed results stay Pending until each test is run. No results are claimed in this deck.

Appendix A3 · Evaluation & Triage Baseline

What will be measured, and how manual and tool-assisted triage will be compared

Measure	How it will be taken
Processing time	Timer around import-to-timeline per scenario; machine noted
Timeline completeness	Events in timeline vs events in the ground-truth list
Integrity-check results	Pass or fail per file; modified test file detected or not
Parsing gaps	Injected bad lines vs lines reported as gaps
Duplicate handling	Injected repeats vs groups created; originals intact
Triage time	Minutes to answer scripted questions, manual vs tool

Triage-time record (minutes)

Scenario	Manual	Tool-assisted
S1	Pending	Pending
S2	Pending	Pending
S3	Pending	Pending

Baseline protocol

- Same three scenarios and the same scripted questions for both methods, for example: when did the first failed login occur, which account changed role, what was uploaded and where
- Manual: raw files in a text editor, grep and a spreadsheet; tool-assisted: import, then timeline viewer; timer starts at the first question
- Alternate order and reviewer between runs to limit learning effects; report each run and the median. Small sample, so no general claim

Appendix A4 · Limitations & Security

Known limits and how the design handles them

Challenge	Mitigation and honest limit
Many log formats	Two formats required, four targeted; others via parser modules. Unsupported lines are logged as parsing gaps
Clock drift, missing zone	Keep raw timestamps; record assumptions; analyst enters known per-source skew [2]. Unknown skew cannot be detected automatically
Hash-only integrity	SHA-256 detects changes relative to a hash recorded earlier. It does not prove who collected the data or give an unalterable chain of custody; someone who can alter both file and manifest can evade it. Signed or exported manifests are future scope
Heuristic correlation	De-duplication and grouping can be wrong; duplicates are grouped, never deleted, and the analyst reviews
Sensitive evidence	Local processing; minimize copies; synthetic data for the demo; no real logs sent to external services
False AI interpretation	Optional layer; output labeled draft; analyst validation required
Scope	Prototype for small teams, to be tested on synthetic data only. Not a forensic suite or SIEM; no forensic certification or legal admissibility is claimed

Thank You for Your Attention!

A3-PS013-TC015 · Incident Timeline & Evidence Collection for Small Teams

Barath C G · Sri Ramanarayana S N · Mentor: Dr. Rakoth Kandan Sambandam